Bayesian Optimization as a Budget-Aware Local Optimizer within a Continuous EDA

Aimar Barrena Pol

Optimization Techniques

Abstract. This work evaluates Bayesian Optimization (BO) as a local-search operator within a continuous Estimation of Distribution Algorithm (EDA). A univariate Gaussian EDA is compared with two hybrids using Local Search with L-BFGS-B and Gaussian-process BO with Expected Improvement. All methods are evaluated on Sphere, Rastrigin, Ackley, and Rosenbrock in five dimensions, using ten seeds and exactly 600 objective evaluations per run. Local refinement is restricted to the best population member and receives at most ten evaluations per generation. The plain EDA achieves the lowest median final objective on all four benchmarks. BO improves on the median performance of the L-BFGS-B hybrid on three benchmarks, but does not compensate consistently for the population evaluations it replaces. These results show that adding surrogate-guided refinement is not automatically beneficial under a small shared evaluation budget. The conclusions concern this specific short-budget implementation; they do not establish a general ranking of local optimizers.

Keywords: Estimation of Distribution Algorithms · Bayesian Optimization · Local search · Evaluation budget

1 Problem Definition

This work investigates whether Bayesian Optimization (BO) can improve the performance of a continuous Estimation of Distribution Algorithm (EDA) when used as a local-search operator. The problem is to find high-quality solutions within a fixed number of objective evaluations.

1.1 Optimization Setting

We consider bounded continuous minimization problems of the form

$$\min_{\mathbf{x}\in\mathcal{X}} f(\mathbf{x}), \qquad \mathcal{X} = \prod_{j=1}^{d} [\ell_j, u_j], \tag{1}$$

where $\mathbf{x}$ is a vector of d real-valued decision variables, f is the objective function, and ℓ_j and u_j define the bounds of each variable. The algorithms obtain information about the objective through evaluations of candidate solutions. Given a

budget B, the desired output is the best solution observed before that budget is exhausted:

$$\hat{\mathbf{x}}_B \in \underset{\mathbf{x}\in\{\mathbf{x}_1,\ldots,\mathbf{x}_N\}}{\arg\min} f(\mathbf{x}), \qquad N \leq B. \tag{2}$$

The budget includes evaluations used by both population search and local search, including numerical gradient estimation when applicable.

The experimental scope is restricted to four deterministic benchmark functions in five dimensions: Sphere, Rastrigin, Ackley, and Rosenbrock. These provide different search landscapes on which to examine the proposed algorithms.

1.2 Motivation for Local Refinement

EDAs learn a probability distribution from selected solutions and sample new candidates from it [2]. The baseline uses independent Gaussian marginals, which concentrate sampling around promising regions but do not represent dependencies between variables. Local refinement may improve precision, but it also consumes additional evaluations that could be used for population search. BO adds a surrogate that can guide these local evaluations, while reducing the need for expensive function evaluations compared to traditional methods. The experiment tests whether that tradeoff is favorable.

1.3 Research Objective and Comparison

The central research question is whether BO-based local refinement produces better solutions within the same objective-evaluation budget than either a plain Gaussian EDA or an EDA using L-BFGS-B local search. The plain EDA provides the population-search baseline, while L-BFGS-B provides a conventional local-optimization comparison. Together, these methods allow the experiments to assess both the value of adding local search and the relative performance of the two local-search choices.

Performance is examined through the final best objective value and variability across repeated runs. Evaluation histories are additionally used to compare convergence of the models. Equal evaluation budgets are the principal comparison constraint. Conclusions are limited to the selected benchmarks, dimensionality, and experimental settings, rather than establishing that one approach is universally preferable.

2 Steps for Solving the Problem

2.1 Gaussian EDA as the Population Search

The implementation uses the `GaussianUMDA` interface from `pateda`. Its search model is a product of univariate normal distributions. If S_t denotes the selected individuals in generation t, the basic model can be expressed as

$$p_t(\mathbf{x}) = \prod_{j=1}^{d} \mathcal{N}(x_j; \mu_{t,j}, \sigma_{t,j}^2), \quad \mu_{t,j} = \frac{1}{|S_t|} \sum_{\mathbf{x}\in S_t} x_j, \tag{3}$$

where each variance describes the spread of the selected values in one coordinate. This factorization is computationally simple, but it cannot represent correlations between coordinates. That limitation is relevant on landscapes such as Rosenbrock, where good solutions follow a curved valley.

Each run uses a population of 30, selects the best half, and enables elitism. The initial population and subsequent generations are managed by the EDA framework within the specified bounds. Because the framework maximizes fitness, the minimization objective is supplied as $F(\mathbf{x}) = -f(\mathbf{x})$. All reported results are converted back to the original objective, so smaller values always indicate better solutions. The plain method evaluates its initial population and 19 subsequent populations, consuming $30 + 19 \times 30 = 600$ evaluations.

2.2 Shared Local-Search Interface

The two hybrids retain the same population model, selection ratio, population size, and elitism setting. A local-search component receives a population and its evaluated fitness values, selects the member with the highest fitness, and attempts to improve it. The selected individual is replaced only if a strictly better evaluated candidate is found. Its original fitness is retained otherwise. This rule prevents a local call from directly worsening that individual, although the hybrid can still finish worse than the plain EDA because it allocates evaluations differently.

Both hybrids request 14 generations after initialization, using

$$G_{\text{hybrid}} = \left\lceil \frac{600}{30 + 10} \right\rceil - 1 = 14. \tag{4}$$

The local component is attached through the framework's internal component interface. Each invocation receives the remaining global budget and is capped at ten evaluations, both per generation and per selected solution. With only one selected solution, these two local limits coincide. The saved results confirm total use of 600 evaluations in every hybrid run. They do not separately record the number spent in the population and local components, so no measured breakdown between those components is claimed.

A counter wraps the true objective and is shared by the population and local searches. Every true call increments it, and a call after the global limit raises an error. Consequently, numerical differentiation and repeated evaluations count toward the same budget. Surrogate fitting and acquisition optimization do not call the benchmark objective and therefore do not consume this budget, although they consume computation time.

2.3 Classical Refinement with L-BFGS-B

The classical operator calls `scipy.optimize.minimize` with the L-BFGS-B method and the original global bounds. L-BFGS-B is a bound-constrained quasi-Newton method. No analytic Jacobian is supplied, so gradients are estimated

numerically. The implementation sets both `maxiter` and `maxfun` to the available local budget. These parameters alone are insufficient to enforce a strict objective-call limit when numerical gradients are used, as documented by SciPy [4].

An additional guard therefore returns a large penalty, 10^{100}, after ten true local evaluations have been consumed. These subsequent penalty returns do not evaluate the benchmark. The wrapper tracks the best actual point visited and returns that point, rather than relying on the optimizer's final status or final iterate. This protects the evaluation budget but can interrupt a gradient estimate or line search. Accordingly, the experiment evaluates a tightly budgeted L-BFGS-B wrapper, rather than L-BFGS-B run to convergence.

In five dimensions, a forward-difference gradient can require approximately six objective calls including its base point, leaving little room for line searches. The classical wrapper charges repeated starting-point evaluations to its budget, whereas BO reuses the known centre.

2.4 Bayesian Refinement with Expected Improvement

BO constructs a probabilistic surrogate from evaluated points and uses an acquisition function to choose the next evaluation [1]. Here each local call creates a new `skopt.Optimizer` with a Gaussian-process surrogate and Expected Improvement (EI). Gaussian processes provide predictive means and variances from a covariance model [3]. The notebook leaves the detailed covariance and fitting settings at the library defaults; these are not independently tuned in this study.

The search region is a fixed box around the selected centre $\mathbf{c}$:

$$a_j = \max\{\ell_j, c_j - 0.05(u_j - \ell_j)\}, \qquad b_j = \min\{u_j, c_j + 0.05(u_j - \ell_j)\}. \tag{5}$$

Thus, 5% is the radius along each coordinate. Away from a global boundary, the full local width is 10% of the global range. This is a fixed local box, not an adaptive trust-region algorithm.

The already evaluated centre and its objective value initialize the optimizer through `tell`; no extra benchmark call is needed. Setting `n_initial_points=1` makes this observation satisfy the initial-design requirement. Each of the next ten iterations asks for a candidate, evaluates its true objective, and updates the surrogate. For minimization, EI can be written as

$$\mathrm{EI}(\mathbf{x}) = \mathbb{E}[\max\{0, f_{\min} - f(\mathbf{x}) - \xi\} \mid \mathcal{D}], \tag{6}$$

where $\mathcal{D}$ contains the observations, $f_{\min}$ is the best observed local value, and ξ is the acquisition's exploration parameter. The implementation uses the library's default acquisition parameters. A new surrogate is created at every local invocation, so previous generations' observations are not reused. Each completed call therefore has only the centre and at most ten newly evaluated observations available to guide refinement.

3 Experimental Design

3.1 Benchmark Suite

The experiments use four deterministic continuous functions in $d = 5$ dimensions. The conventional definitions are given below, with the scalar domain bounds applied to every coordinate:

$$f_{\text{Sphere}}(\mathbf{x}) = \sum_{j=1}^{d} x_j^2, \tag{7}$$

$$f_{\text{Rastrigin}}(\mathbf{x}) = 10d + \sum_{j=1}^{d} \left[x_j^2 - 10\cos(2\pi x_j) \right], \tag{8}$$

$$f_{\text{Ackley}}(\mathbf{x}) = -20\exp\left(-0.2\sqrt{\frac{1}{d}\sum_{j=1}^{d} x_j^2} \right) - \exp\left(\frac{1}{d}\sum_{j=1}^{d} \cos(2\pi x_j) \right) + 20 + e, \tag{9}$$

$$f_{\text{Rosenbrock}}(\mathbf{x}) = \sum_{j=1}^{d-1} \left[100(x_{j+1} - x_j^2)^2 + (1 - x_j)^2 \right]. \tag{10}$$

The notebook imports the benchmark implementations from `pateda`. Sphere and Rastrigin use $[-5.12, 5.12]^5$, Ackley uses $[-32.768, 32.768]^5$, and Rosenbrock uses $[-5, 10]^5$. The global minimum is zero for each conventional function: at the origin for the first three, and at the all-ones vector for Rosenbrock. Final objective values can consequently be read as residual errors relative to these optima.

Sphere is a smooth separable baseline. Rastrigin introduces many local minima while retaining coordinate separability. Ackley combines oscillations with a central basin and a broad outer region. Rosenbrock introduces strong interactions between adjacent variables and a narrow curved valley. Together they test different difficulties, but they do not represent a broad survey of continuous optimization problems.

3.2 Controlled Settings and Repetition

Table 1 summarizes the common settings and the differences introduced by the local operators. There are ten runs for each method–benchmark combination, using seeds 0 through 9. This gives 120 runs in total and 72,000 recorded objective evaluations. All 120 CSV records contain exactly 600 evaluations. The comparisons therefore use the same realized evaluation budget, rather than merely the same nominal cap.

Identical seed labels support descriptive comparisons but do not imply identical candidate populations after the methods diverge. No hyperparameter sweep is included in the saved experiment.

Table 1. Experimental settings used in the notebook.

Setting	Value
Decision variables	5 continuous variables
Population and selection	30 individuals; best 50% selected
Elitism	Enabled for all three methods
Objective budget	600 true evaluations per run
Repetitions	10 seeds per benchmark and method
Local subset	Best individual in the population
Local allowance	At most 10 evaluations per invocation
Classical search	L-BFGS-B; numerical gradients; global bounds
BO search	Gaussian process; EI; fixed local box
BO box radius	5% of the global range per coordinate

3.3 Performance Measures and Available Evidence

The main measure is the final best objective value. For each benchmark and method, the median summarizes the ten runs, and the first and third quartiles describe their dispersion. Quartiles use linear interpolation, matching the notebook's `pandas` calculations. Values from different functions are not pooled because their scales and landscapes differ. A logarithmic vertical axis is used for within-benchmark distribution plots; all saved final values are positive.

Pairwise wins count strictly lower final objectives for corresponding seed labels, with denominator ten. A baseline win means beating the plain EDA, rather than being best among all three methods.

The CSV files store endpoints rather than evaluation-by-evaluation histories. They support final-value comparisons but cannot establish convergence speed, the stage at which improvements occurred, or the number of successful local calls. Existing convergence or ablation graphics are therefore not used as evidence for this experiment. Runtime is also absent, so the analysis concerns objective-evaluation efficiency rather than elapsed-time efficiency. The experiment is reported descriptively, without claims of statistical significance from ten runs per configuration.

3.4 Reproducibility

The source notebook contains the algorithms and the loops that save the three method-specific CSV files. Report tables and the distribution figure are generated directly from these files. The software stack includes Python, NumPy, pandas, SciPy, scikit-optimize, and `pateda`. The repository does not provide a complete locked dependency environment, which limits exact reproduction across library versions, especially for default Gaussian-process settings and internal EDA interfaces. This report analyzes the saved runs and does not claim to have rerun the optimization experiments.

Table 2. Final objective after 600 evaluations. Entries show median and, underneath, $[Q_1, Q_3]$ across ten runs. Lower is better; bold medians identify the best method per benchmark.

Benchmark	Plain EDA	EDA + L-BFGS-B	EDA + BO
Sphere	$\mathbf{3.55\times10^{-5}}$	5.02×10^{-4}	3.12×10^{-4}
	$[9.68\times10^{-6}, 7.38\times10^{-5}]$	$[3.81\times10^{-4}, 6.17\times10^{-4}]$	$[2.36\times10^{-4}, 6.45\times10^{-4}]$
Rastrigin	**6.44**	8.11	7.44
	$[4.81, 9.2]$	$[7.17, 11.4]$	$[5.64, 8.8]$
Ackley	**0.0897**	0.417	0.484
	$[0.0486, 0.0999]$	$[0.253, 0.804]$	$[0.388, 0.525]$
Rosenbrock	**4.61**	7.16	5.49
	$[3.44, 6.01]$	$[3.82, 9.74]$	$[3.56, 6.83]$

4 Results of the Experiments

4.1 Final Objective Values

Table 2 reports the medians and interquartile ranges from the current CSV files. The plain EDA has the lowest median on all four benchmarks. The BO hybrid has a lower median than the L-BFGS-B hybrid on Sphere, Rastrigin, and Rosenbrock, while L-BFGS-B has the lower median on Ackley. Thus, the results give some support for BO relative to this particular classical wrapper, but no median improvement over the plain population method.

On Sphere, the plain EDA reaches a median of 3.55×10^{-5}, compared with 3.12×10^{-4} for BO and 5.02×10^{-4} for L-BFGS-B. The BO median residual is approximately 8.8 times the plain median, and the classical hybrid's is approximately 14.2 times as large. Both hybrids reach small absolute errors, but the extra local operator does not improve precision at the fixed evaluation budget. The BO results also include a relatively poor run at 3.83×10^{-3}, whereas its best run reaches 2.51×10^{-5}.

On Rastrigin, the medians are 6.44 for the plain EDA, 7.44 for BO, and 8.11 for L-BFGS-B. The BO interquartile interval, $[5.64, 8.80]$, overlaps the plain EDA interval, $[4.81, 9.20]$. The relatively small difference in medians and substantial run variation make this a less consistent separation than Sphere or Ackley. BO performs competitively on some seeds, but the observed endpoints do not show reliable escape from the function's local minima.

Ackley produces the clearest baseline advantage. The plain EDA median is 0.0897, compared with 0.417 for L-BFGS-B and 0.484 for BO. The corresponding hybrid-to-plain median ratios are approximately 4.7 and 5.4. The interquartile intervals are separated: the upper quartile of the plain EDA is about 0.100, below the lower quartile of either hybrid. Both local methods therefore use the budget less effectively on this benchmark in the observed experiment.

On Rosenbrock, the plain EDA median is 4.61, BO reaches 5.49, and L-BFGS-B reaches 7.16. The distributions overlap considerably. BO's median is

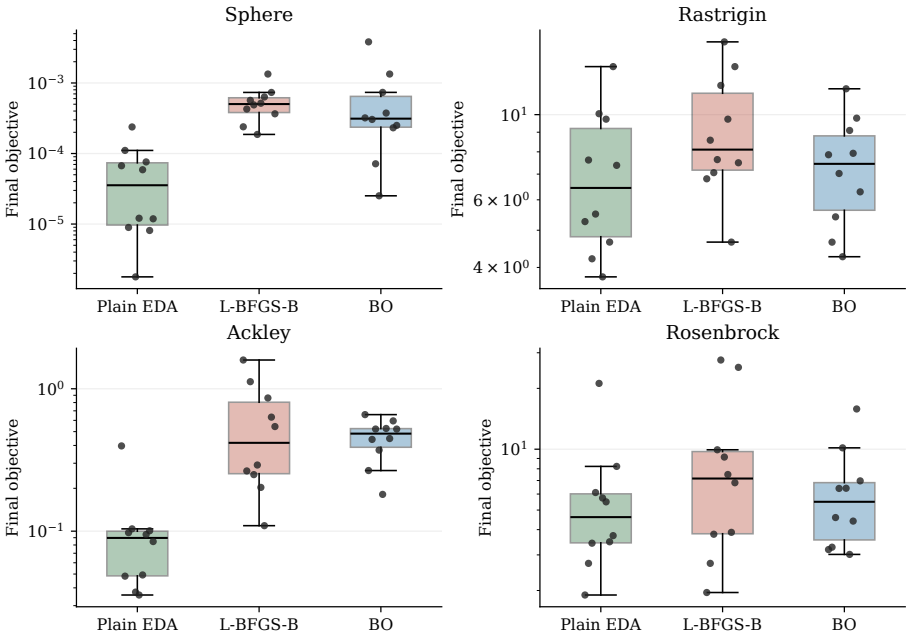


Fig. 1. Final objective distributions from the current saved runs. Each dot is one run; boxes show the interquartile range and median, with whiskers extending to the most extreme observations within 1.5 interquartile ranges. Each panel uses its own logarithmic vertical scale.

approximately 19% above the baseline, while the classical hybrid's is approximately 55% above it. No method consistently approaches the zero optimum in this setting. The coordinate-independent population model and the short local calls are plausible constraints on progress along the curved valley, although the endpoint records cannot isolate their individual effects.

4.2 Comparisons by Seed

Table 3 complements the medians with direct comparisons between corresponding seed labels. BO beats the plain EDA in one of ten Sphere runs, five Rastrigin runs, no Ackley runs, and three Rosenbrock runs. The classical hybrid beats the plain EDA in zero, three, zero, and one run, respectively. BO therefore records more baseline wins than the classical hybrid on three functions and ties it on Ackley.

On Ackley, BO beats L-BFGS-B on six corresponding seeds despite having a worse marginal median. Likewise, BO wins half the Rastrigin baseline comparisons but has a worse median. Pairwise counts and marginal medians describe different aspects of the sample.

Table 3. Strict pairwise wins out of ten corresponding seed labels. A win means a lower final objective for the method listed first.

Benchmark	BO vs. plain	L-BFGS-B vs. plain	BO vs. L-BFGS-B
Sphere	1	0	6
Rastrigin	5	3	8
Ackley	0	0	6
Rosenbrock	3	1	5

4.3 Interpretation of the Budget Tradeoff

The consistent median advantage of the plain EDA suggests that, for these settings, keeping more evaluations in population search is more useful than repeatedly refining one individual. The plain method is configured for 19 subsequent populations, while each hybrid requests 14. This reduces opportunities to select and refit the population model. A successful local improvement must offset that change in search trajectory as well as its direct evaluation cost.

BO has additional restrictions. Its model is restarted around one centre at every invocation, giving it at most eleven observations in five dimensions. The box radius remains fixed as the population contracts, so the local region does not become smaller automatically when greater precision is needed. The physical radius also varies with the benchmark domain: 0.512 per coordinate on Sphere and Rastrigin, 3.2768 on Ackley, and 0.75 on Rosenbrock before boundary clipping. Consequently, the same relative parameter does not imply the same relationship to local landscape structure.

L-BFGS-B faces a different constraint: numerical gradients and line-search evaluations compete for ten true calls. The artificial-penalty guard can stop useful optimizer work partway through a numerical step. Furthermore, BO is restricted to a local box whereas L-BFGS-B uses the full global bounds. The comparison therefore concerns complete implemented operators, including their domains, initialization, and budget handling. It does not isolate the effect of substituting one optimizer while holding every other local-search detail identical.

These mechanisms provide possible explanations, rather than measured causal decompositions. Without local-call logs, histories, and controlled ablations, the saved endpoints cannot determine which mechanism accounts for the observed differences. What they do establish is that neither hybrid improves the median final outcome over the plain EDA under the recorded budget.

5 Conclusions

This study implemented and compared a plain Gaussian EDA, an EDA with tightly budgeted L-BFGS-B refinement, and an EDA with Gaussian-process BO refinement. Across four five-dimensional benchmarks and ten seeds per configuration, every run used exactly 600 objective evaluations. The plain EDA achieved the lowest median final objective on every benchmark. The central

research question therefore receives a negative answer for the tested configuration: BO-based local refinement did not improve median evaluation efficiency over allocating the budget to population search.

BO nevertheless compared favorably with the classical hybrid on three of the four marginal medians and won more corresponding-seed comparisons against L-BFGS-B on Sphere, Rastrigin, and Ackley. These results support a limited conclusion about the implemented alternatives, rather than a general superiority claim for BO. The classical optimizer had little budget for numerical gradients, and the two operators differed in their search bounds and reuse of the starting evaluation.

The most useful extension would record the best-so-far objective after every true evaluation, the cost and improvement of each local invocation, and elapsed time. Controlled experiments could then vary local-search frequency and budget, compare equal local domains, reuse nearby observations across BO calls, and adapt the local radius to population spread. A classical baseline with enough evaluations to complete numerical-gradient steps, or with analytic derivatives when available, would clarify whether its current weakness is caused mainly by the ten-call limit. Larger budgets and additional dimensions would test whether the observed ranking persists. Until such evidence is available, the plain Gaussian EDA is the best-supported choice among the three methods for the settings studied here.

References

1. Frazier, P.I.: A tutorial on bayesian optimization. arXiv preprint arXiv:1807.02811 (2018). https://doi.org/10.48550/arXiv.1807.02811
2. Larrañaga, P., Lozano, J.A. (eds.): Estimation of Distribution Algorithms: A New Tool for Evolutionary Computation. Kluwer Academic Publishers (2002)
3. Rasmussen, C.E., Williams, C.K.I.: Gaussian Processes for Machine Learning. MIT Press (2006), https://gaussianprocess.org/gpml/
4. SciPy Developers: minimize with method l-bfgs-b. SciPy optimization documentation, https://docs.scipy.org/doc/scipy/reference/optimize.minimize-lbfgsb.html, accessed 29 October 2026